# #

→ Variable rule
→ Building url dynamically
→ Jinja 2 template engine

(1) Variable rule

~~@ app.route ('/submit~~

parameter passed

```
@app.route('/success/<score>')
def success(score):
    return f'You scored {score}
```

using it

Note — To limit the datatyhe of the variable,

```
@app.route('/success/<int:score>')
```

datatyhe

variable name

will only allow integer type input

This method is used to pass a variable directly ('GET' method or request.method = 'GET') by typing to address bar.

Note → Another way to utilise this is by using redirect (url_for('...', para = var...))

Jinja 2 — template engine

Passing a value to an html page (using render_template)

```
>>> @app.route('/success/<int:score>')
def success(score):
    return render_template('abc.html', marks=score)
```

abc.html, where the variable will be passed

marks = the variable name that abc.html will look for

How? to recieve

{{ ... }}

html

```
<h1>
    The score you got is {{ marks }}
</h1>
```

Same variable name as passed with render_template

use double braces

VVV imp

Jinja2 template engine

{{ . . . }} → expressions to print/ ~~output in~~ use passed variables via html.

{% . . . . . %} → to write conditions or loops in html

{# . . . #} → to write comments in html.

---

eg for a dictionary being passed as

>>> return render_template('abc.html', res = dict1)

variable name — dictionary

html

```
{% for key, value in res.items() %}
{{ key }}
{{ value }}
{% endfor %}
{# You need to endfor in html #}
```

a for loop defination — using items from dict1 — ending for loop — comment example

~~of if~~

revision

(A) to specify / check methods.

>>> @app.route('/abc', methods = ['GET', 'POST'])

>>> if request.method == 'GET':

(A) to pass a value to html.

render_template('abc.html', ref = variable)

use this in html to access passed variable

(A) To use the variable in html, use Jinja2 template engine.

{{ ... }} , {% ... %}, {# ... #}

(A) Always import Flask, render_template, request from flask

(A) To retrieve value from html in code,

x = request.form['id/name']

id/name from html page of required field.

Building url dynamically

redirect, url_for

```
>>> from flask import Flask, request, redirect,
                      url_for.
```

```
@app.route('/home')
def home():
    pass
    return 'Hello'
```

XX must have a return

```
@app.route('/submit')
def submit():
    if request.method == 'GET':
        return redirect(url_for('home'))
```

parameter for url

redirect() → redirects to the url passed inside as parameter

url_for → dynamically builds a url for the passed parameter.

in this case, \home route exists, which is obviously necessary for ~~redirect(~~ this block to work, so the user will be taken to the \home route if method == 'GET'.

You can also pass variables while redirecting if the route already accepts variables.

eg

```
>>> @app.route ('/home/<int:score>')
    def home (score):
        return score

>>> @app.route ('/submit')
    def submit ():
        total = 50
        return redirect (url_for ('home', score = total))
```

Here, a 2nd parameter passed into url_for( ) serves as a variable to be ingested by /home directory, which takes 'int' type input already.

VVV imp